04/09/2026: Score-based generative modeling through SDEs

Reference: Song et al. (2021) *Score-based generative modeling through SDEs.*

CSCI 546: Diffusion Models

Table of contents

1. Course Logistics
2. Mini-Lecture
3. Lightning Chat
4. Tempest Overview

Course Logistics

Announcements

- I need to leave right when class ends today to give a short talk in another class at 12:15.
- I will be in office hours (Barnard 352) 12:30-2 if you'd like to chat.

Reading Survey #6

1. This paper is about score-based generative modeling via SDEs. Describe the relationship between score-based generative modeling and SDEs.

Mini-Lecture

Big Question:

How is score matching related to diffusion models?

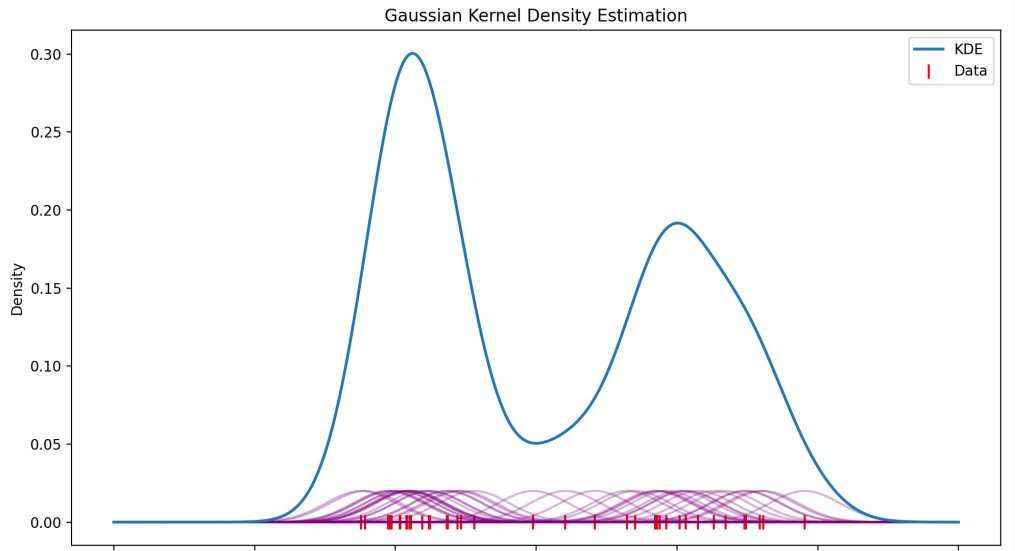
Kernel Density Estimates

Given observed data $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N$, we can form a Kernel Density Estimator (KDE) for the true density $p(\boldsymbol{x})$

$$\underbrace{q_\sigma(\boldsymbol{z})}_{\text{KDE}} = \int \underbrace{q_\sigma(\boldsymbol{z} \mid \boldsymbol{x})}_{\text{kernel}} \underbrace{p(\boldsymbol{x})}_{\text{true density}} \, \mathrm{d}x$$

$$\approx \frac{1}{N} \sum_{n=1}^{N} q_\sigma(\boldsymbol{z} \mid \boldsymbol{x}_n) \qquad \text{where } \boldsymbol{x}_n \sim p(\boldsymbol{x})$$

With a Gaussian Kernel $q_\sigma(\boldsymbol{z} \mid \boldsymbol{x}_n) = \mathcal{N}(\boldsymbol{x}_n, \sigma^2 \boldsymbol{I})$,



From score matching to diffusion

Using a Gaussian KDE, the score-matching loss function for a neural network with weights $\boldsymbol{w}$ becomes:

$$J(\boldsymbol{w}) = \frac{1}{2}\int \Big|\Big| \underbrace{\boldsymbol{s}_{\boldsymbol{w}}(\boldsymbol{z})}_{\text{estimated score}} - \underbrace{\nabla_{\boldsymbol{z}} \ln q_\sigma(\boldsymbol{z})}_{\text{“true” score}} \Big|\Big|^2 \underbrace{q_\sigma(\boldsymbol{z})}_{\text{KDE}} \, \mathrm{d}\boldsymbol{z}$$

From score matching to diffusion

Using a Gaussian KDE, the score-matching loss function for a neural network with weights $\boldsymbol{w}$ becomes:

$$J(\boldsymbol{w}) = \frac{1}{2}\int \Big|\Big| \underbrace{\boldsymbol{s}_{\boldsymbol{w}}(\boldsymbol{z})}_{\text{estimated score}} - \underbrace{\nabla_{\boldsymbol{z}} \ln q_\sigma(\boldsymbol{z})}_{\text{“true” score}} \Big|\Big|^2 \underbrace{q_\sigma(\boldsymbol{z})}_{\text{KDE}} \, \mathrm{d}\boldsymbol{z}$$

$$= \frac{1}{2}\int\int \Big|\Big| \boldsymbol{s}_{\boldsymbol{w}}(\boldsymbol{z}) - \underbrace{\nabla_{\boldsymbol{z}} \ln q_\sigma(\boldsymbol{z} \mid \boldsymbol{x})}_{\text{kernel score}} \Big|\Big|^2 \underbrace{q_\sigma(\boldsymbol{z} \mid \boldsymbol{x})}_{\text{kernel}} \underbrace{p(\boldsymbol{x})}_{\text{true density}} \, \mathrm{d}\boldsymbol{x}\mathrm{d}\boldsymbol{z}$$

From score matching to diffusion

Using a Gaussian KDE, the score-matching loss function for a neural network with weights $\boldsymbol{w}$ becomes:

$$J(\boldsymbol{w}) = \frac{1}{2}\int \Big|\Big| \underbrace{\boldsymbol{s}_{\boldsymbol{w}}(\boldsymbol{z})}_{\text{estimated score}} - \underbrace{\nabla_{\boldsymbol{z}} \ln q_\sigma(\boldsymbol{z})}_{\text{“true” score}} \Big|\Big|^2 \underbrace{q_\sigma(\boldsymbol{z})}_{\text{KDE}} \, \mathrm{d}\boldsymbol{z}$$

$$= \frac{1}{2}\int\int \Big|\Big| \boldsymbol{s}_{\boldsymbol{w}}(\boldsymbol{z}) - \underbrace{\nabla_{\boldsymbol{z}} \ln q_\sigma(\boldsymbol{z} \mid \boldsymbol{x})}_{\text{kernel score}} \Big|\Big|^2 \underbrace{q_\sigma(\boldsymbol{z} \mid \boldsymbol{x})}_{\text{kernel}} \underbrace{p(\boldsymbol{x})}_{\text{true density}} \, \mathrm{d}\boldsymbol{x}\mathrm{d}\boldsymbol{z}$$

$$\approx \frac{1}{2N}\sum_{n=1}^{N}\int \Big|\Big| \boldsymbol{s}_{\boldsymbol{w}}(\boldsymbol{z}) - \underbrace{\nabla_{\boldsymbol{z}} \ln q_\sigma(\boldsymbol{z} \mid \boldsymbol{x}_n)}_{\text{kernel score}} \Big|\Big|^2 \underbrace{q_\sigma(\boldsymbol{z} \mid \boldsymbol{x}_n)}_{\text{kernel}} \, \mathrm{d}\boldsymbol{z}$$

$$\text{where } \boldsymbol{x}_n \sim \underbrace{p(\boldsymbol{x})}_{\text{true density}}$$

From score matching to diffusion

Using a Gaussian KDE, the score-matching loss function for a neural network with weights $\boldsymbol{w}$ becomes:

$$
\begin{aligned}
J(\boldsymbol{w}) &= \frac{1}{2}\int \Big|\Big| \underbrace{\boldsymbol{s}_{\boldsymbol{w}}(\boldsymbol{z})}_{\text{estimated score}} - \underbrace{\nabla_{\boldsymbol{z}} \ln q_\sigma(\boldsymbol{z})}_{\text{“true” score}} \Big|\Big|^2 \underbrace{q_\sigma(\boldsymbol{z})}_{\text{KDE}} \, \mathrm{d}\boldsymbol{z} \\
&= \frac{1}{2}\int\int \Big|\Big| \boldsymbol{s}_{\boldsymbol{w}}(\boldsymbol{z}) - \underbrace{\nabla_{\boldsymbol{z}} \ln q_\sigma(\boldsymbol{z} \mid \boldsymbol{x})}_{\text{kernel score}} \Big|\Big|^2 \underbrace{q_\sigma(\boldsymbol{z} \mid \boldsymbol{x})}_{\text{kernel}} \underbrace{p(\boldsymbol{x})}_{\text{true density}} \, \mathrm{d}\boldsymbol{x}\mathrm{d}\boldsymbol{z} \\
&\approx \frac{1}{2N}\sum_{n=1}^{N}\int \Big|\Big| \boldsymbol{s}_{\boldsymbol{w}}(\boldsymbol{z}) - \underbrace{\nabla_{\boldsymbol{z}} \ln q_\sigma(\boldsymbol{z} \mid \boldsymbol{x}_n)}_{\text{kernel score}} \Big|\Big|^2 \underbrace{q_\sigma(\boldsymbol{z} \mid \boldsymbol{x}_n)}_{\text{kernel}} \, \mathrm{d}\boldsymbol{z} \\
&\qquad \text{where } \boldsymbol{x}_n \sim \underbrace{p(\boldsymbol{x})}_{\text{true density}}
\end{aligned}
$$

Remarks. All equalities are up to a constant in $\boldsymbol{w}$. The second equality is from Vincent (2011). The third is Monte Carlo approximation. Reference: Bishop Sec 20.3.2.

From score matching to diffusion

Using a Gaussian KDE, the score-matching loss function for a neural network with weights $\boldsymbol{w}$ becomes:

$$J(\boldsymbol{w}) = \frac{1}{2}\int \Big|\Big| \underbrace{\boldsymbol{s}_{\boldsymbol{w}}(\boldsymbol{z})}_{\text{estimated score}} - \underbrace{\nabla_{\boldsymbol{z}} \ln q_\sigma(\boldsymbol{z})}_{\text{“true” score}} \Big|\Big|^2 \underbrace{q_\sigma(\boldsymbol{z})}_{\text{KDE}} \, \mathrm{d}\boldsymbol{z}$$

$$\approx \frac{1}{2N}\sum_{n=1}^{N}\int \Big|\Big| \boldsymbol{s}_{\boldsymbol{w}}(\boldsymbol{z}) - \underbrace{\nabla_{\boldsymbol{z}} \ln q_\sigma(\boldsymbol{z} \mid \boldsymbol{x}_n)}_{\text{kernel score}} \Big|\Big|^2 \underbrace{q_\sigma(\boldsymbol{z} \mid \boldsymbol{x}_n)}_{\text{kernel}} \, \mathrm{d}\boldsymbol{z}$$

$$\text{where } \boldsymbol{x}_n \sim \underbrace{p(\boldsymbol{x})}_{\text{true density}}$$

Now compute $\nabla_{\boldsymbol{z}} \ln q_\sigma(\boldsymbol{z} \mid \boldsymbol{x}_n)$ under a diffusion model, whose multistep kernel is $q_\sigma(\boldsymbol{z} \mid \boldsymbol{x}_n) = \mathcal{N}\big(\sqrt{\alpha_t}\boldsymbol{x}_n, (1-\alpha_t)\boldsymbol{I}\big)$.

From score matching to diffusion

Using a Gaussian KDE, the score-matching loss function for a neural network with weights $\boldsymbol{w}$ becomes:

$$J(\boldsymbol{w}) = \frac{1}{2}\int \Big|\Big| \underbrace{\boldsymbol{s}_{\boldsymbol{w}}(\boldsymbol{z})}_{\text{estimated score}} - \underbrace{\nabla_{\boldsymbol{z}} \ln q_\sigma(\boldsymbol{z})}_{\text{“true” score}} \Big|\Big|^2 \underbrace{q_\sigma(\boldsymbol{z})}_{\text{KDE}} \, \mathrm{d}\boldsymbol{z}$$

$$\approx \frac{1}{2N}\sum_{n=1}^{N}\int \Big|\Big| \boldsymbol{s}_{\boldsymbol{w}}(\boldsymbol{z}) - \underbrace{\nabla_{\boldsymbol{z}} \ln q_\sigma(\boldsymbol{z} \mid \boldsymbol{x}_n)}_{\text{kernel score}} \Big|\Big|^2 \underbrace{q_\sigma(\boldsymbol{z} \mid \boldsymbol{x}_n)}_{\text{kernel}} \, \mathrm{d}\boldsymbol{z}$$

$$\text{where } \boldsymbol{x}_n \sim \underbrace{p(\boldsymbol{x})}_{\text{true density}}$$

Now compute $\nabla_{\boldsymbol{z}} \ln q_\sigma(\boldsymbol{z} \mid \boldsymbol{x}_n)$ under a diffusion model, whose multistep kernel is $q_\sigma(\boldsymbol{z} \mid \boldsymbol{x}_n) = \mathcal{N}\big(\sqrt{\alpha_t}\boldsymbol{x}_n, (1-\alpha_t)\boldsymbol{I}\big)$.

Solution: $\nabla_{\boldsymbol{z}} \ln q_\sigma(\boldsymbol{z} \mid \boldsymbol{x}_n) = -\frac{1}{\sqrt{1-\alpha_t}}\boldsymbol{\epsilon}$, where $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \boldsymbol{I})$.

From score matching to diffusion

Using a Gaussian KDE, the score-matching loss function for a neural network with weights $\boldsymbol{w}$ becomes:

$$J(\boldsymbol{w}) = \frac{1}{2}\int \Big|\Big| \underbrace{\boldsymbol{s}_{\boldsymbol{w}}(\boldsymbol{z})}_{\text{estimated score}} - \underbrace{\nabla_{\boldsymbol{z}} \ln q_\sigma(\boldsymbol{z})}_{\text{“true” score}} \Big|\Big|^2 \underbrace{q_\sigma(\boldsymbol{z})}_{\text{KDE}} \, \mathrm{d}\boldsymbol{z}$$

$$\approx \frac{1}{2N}\sum_{n=1}^{N}\int \Big|\Big| \boldsymbol{s}_{\boldsymbol{w}}(\boldsymbol{z}) - \underbrace{\nabla_{\boldsymbol{z}} \ln q_\sigma(\boldsymbol{z} \mid \boldsymbol{x}_n)}_{\text{kernel score}} \Big|\Big|^2 \underbrace{q_\sigma(\boldsymbol{z} \mid \boldsymbol{x}_n)}_{\text{kernel}} \, \mathrm{d}\boldsymbol{z}$$

$$\text{where } \boldsymbol{x}_n \sim \underbrace{p(\boldsymbol{x})}_{\text{true density}}$$

Now compute $\nabla_{\boldsymbol{z}} \ln q_\sigma(\boldsymbol{z} \mid \boldsymbol{x}_n)$ under a diffusion model, whose multistep kernel is $q_\sigma(\boldsymbol{z} \mid \boldsymbol{x}_n) = \mathcal{N}\big(\sqrt{\alpha_t}\boldsymbol{x}_n, (1-\alpha_t)\boldsymbol{I}\big)$.

Solution: $\nabla_{\boldsymbol{z}} \ln q_\sigma(\boldsymbol{z} \mid \boldsymbol{x}_n) = -\frac{1}{\sqrt{1-\alpha_t}}\boldsymbol{\epsilon}$, where $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \boldsymbol{I})$.

Poll. So what?

From score matching to diffusion

Using a Gaussian KDE, the score-matching loss function for a neural network with weights $\boldsymbol{w}$ becomes:

$$J(\boldsymbol{w}) = \frac{1}{2}\int \Big|\Big| \underbrace{\boldsymbol{s}_{\boldsymbol{w}}(\boldsymbol{z})}_{\text{estimated score}} - \underbrace{\nabla_{\boldsymbol{z}} \ln q_\sigma(\boldsymbol{z})}_{\text{“true” score}} \Big|\Big|^2 \underbrace{q_\sigma(\boldsymbol{z})}_{\text{KDE}} \, \mathrm{d}\boldsymbol{z}$$

$$\approx \frac{1}{2N}\sum_{n=1}^{N}\int \Big|\Big| \boldsymbol{s}_{\boldsymbol{w}}(\boldsymbol{z}) - \underbrace{\nabla_{\boldsymbol{z}} \ln q_\sigma(\boldsymbol{z} \mid \boldsymbol{x}_n)}_{\text{kernel score}} \Big|\Big|^2 \underbrace{q_\sigma(\boldsymbol{z} \mid \boldsymbol{x}_n)}_{\text{kernel}} \, \mathrm{d}\boldsymbol{z}$$

$$\text{where } \boldsymbol{x}_n \sim \underbrace{p(\boldsymbol{x})}_{\text{true density}}$$

Now compute $\nabla_{\boldsymbol{z}} \ln q_\sigma(\boldsymbol{z} \mid \boldsymbol{x}_n)$ under a diffusion model, whose multistep kernel is $q_\sigma(\boldsymbol{z} \mid \boldsymbol{x}_n) = \mathcal{N}\big(\sqrt{\alpha_t}\boldsymbol{x}_n, (1-\alpha_t)\boldsymbol{I}\big)$.

Solution: $\nabla_{\boldsymbol{z}} \ln q_\sigma(\boldsymbol{z} \mid \boldsymbol{x}_n) = -\frac{1}{\sqrt{1-\alpha_t}}\boldsymbol{\epsilon}$, where $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \boldsymbol{I})$.

Poll. So what? We are training the model to **“predict the noise”**, just like diffusion!!!

Big Question:

What are the advantages of bringing in SDE's to generative modeling?

Lightning Chat

Lightning Chat

Tempest Overview

Tempest Overview